1 - Fundamentos

Tabla de contenidos

1 Introducción 1
2 Objetos familiares 1
3 Clases y objetos 2
  3.1 Ejemplo: la clase `dict` 3
4 Creando nuestras propias clases 4
  4.1 Parámetros de inicialización 5
5 Atributos 6
  5.1 Atributos opcionales 8
6 Métodos 9
  6.1 Métodos que modifican estado 10
  6.2 Métodos que reciben argumentos 11
  6.3 Métodos que devuelven resultados 12
  6.4 Métodos que devuelven al objeto 13
7 Ejemplo final: Cuenta bancaria 13

1 Introducción

Para hablar de programación orientada a objetos (OOP, por sus siglas en inglés), podemos empezar preguntándonos qué es un objeto. Aunque no solemos pensar en ello de forma consciente, porque todos podemos distinguir un objeto de lo que no lo es, vale la pena definirlo como algo tangible que se puede percibir, tocar y manipular. ¡Nos pasamos toda la vida interactuando con objetos!

En el mundo real, los objetos tienen atributos o propiedades que los describen. Por ejemplo, un televisor tiene forma, tamaño, color, peso, entre otros. Además, los objetos también pueden realizar acciones. Siguiendo con el ejemplo del televisor, puede encenderse, apagarse, modificar el brillo, la fuente de entrada para ver una serie en Netflix o simplemente reproducir música.

En programación, la definición de objeto no difiere mucho de la anterior. Aunque los objetos no sean elementos físicos, representan entidades que poseen atributos y pueden ejecutar acciones.

La programación orientada a objetos es un paradigma que organiza el código en torno a estos objetos, que combinan datos (atributos) y comportamientos o acciones (métodos) dentro de una misma entidad. Este enfoque permite no solo crear e interactuar con objetos, sino también definir nuestros propios tipos de objetos, adaptados a las necesidades del programa.

2 Objetos familiares

Así como interactuamos con objetos en la vida real desde nuestros primeros días, también lo hemos estado haciendo en Python desde el comienzo. Por ejemplo, cada vez que creamos una cadena de texto, en realidad estamos creando un objeto de la clase `str`.

```
x = "Hola, soy un objeto de tipo 'str'."
y = "¿Y yo? ¡Yo también soy un objeto de tipo 'str'!"
print(f"x = {x}")
print(f"y = {y}")
```

```
x = Hola, soy un objeto de tipo 'str'.
y = ¿Y yo? ¡Yo también soy un objeto de tipo 'str'!
```

Tanto `x` como `y` son objetos del mismo tipo: cadenas de caracteres, que en Python corresponden al tipo `str`. Sin embargo, `x` e `y` no son el mismo objeto, sino dos objetos distintos. Esto puede comprobarse no solo por su contenido (uno de sus atributos), sino también comparando sus identificadores únicos (ID), que son diferentes.

```
print(id(x))
print(id(y))
```

```
140223504929120
140223504962192
```

A su vez, los objetos de tipo `str` pueden realizar ciertas acciones. Por ejemplo, si queremos convertir todas las letras de una cadena a mayúsculas, podemos hacer lo siguiente:

```
x.upper()
```

```
"HOLA, SOY UN OBJETO DE TIPO 'STR'."
```

Las cadenas de caracteres son uno de los muchos tipos de objetos con los que ya hemos interactuado. Todas las estructuras de datos que hemos utilizado —desde las más simples, como los enteros, flotantes y booleanos, hasta las más complejas, como listas, tuplas y diccionarios— no son más que distintos tipos de objetos.

En Python, cada tipo de dato está implementado como una clase, y trabajar con estas clases no solo nos permite crear nuestros propios tipos de datos según nuestras necesidades, sino también interactuar con los objetos que generamos a partir de ellas.

3 Clases y objetos

En el contexto de la programación orientada a objetos, se utilizan **clases** para definir nuevas clases o tipos de objetos, especificando qué atributos deben tener y qué acciones pueden realizar. De forma informal, una clase puede pensarse como una plantilla que determina cómo se ve y se comporta un objeto.

Por ejemplo, imaginemos un molde para hacer budines. Este molde permite producir budines, pero no es un budín en sí. De hecho, se parece más a una fábrica de budines que a un budín.

El molde define aspectos estructurales como la forma y el tamaño de los budines, pero no determina por completo cómo serán. Podemos usarlo para hacer budines según diferentes recetas, bases, colores, aromas, coberturas, etc. y así obtener resultados variados a partir del mismo molde.

En programación orientada a objetos, una clase cumple un rol similar: define la estructura y el comportamiento general de los objetos que se crearán a partir de ella. Los objetos, en cambio, son las instancias concretas generadas a partir de esa clase, cada una con sus propias características particulares, como cada budín que sale del molde.

> **i** ¿Clases o tipos?
>
> En Python, los tipos de datos están implementados como clases. Por eso mismo, en todos los casos debajo, se puede ver que el tipo está precedido por `class`.
>
> ```
> print(type("texto"))
> print(type([1, 2, 3]))
> print(type({"a": 1, "b": 2}))
> ```
>
> ```
> <class 'str'>
> <class 'list'>
> <class 'dict'>
> ```
>
> Así, en Python, es lo mismo hablar de clases o tipos de datos.

3.1 Ejemplo: la clase `dict`

Los diccionarios de Python son instancias de la clase `dict`. Es esta clase la que define, entre otras cosas, que los diccionarios tienen claves y valores.

En el ejemplo debajo, tanto `d1` como `d2` son instancias u objetos de la clase `dict` (es decir, son objetos creados con el mismo molde). Sin embargo, `d1` y `d2` son objetos distintos, que además tienen diferentes valores para sus atributos (claves y valores).

```
d1 = {"a": 100, "b": 250}
d2 = {"m": 20, "n": False}
print(type(d1))
print(type(d2))
```

```
<class 'dict'>
<class 'dict'>
```

```
print(d1 is d2)
print(d1 == d2)
```

```
False
False
```

Si consultamos la ayuda de `dict` mediante `help(dict)`, podemos ver que Python dice que esto se refiere a una clase:

```
help(dict)
```

```
Help on class dict in module builtins:
```

> **i** ¿Objetos o instancias?
>
> En el contexto de programación orientada a objetos, los términos "objeto" e "instancia" suelen usarse de manera intercambiable: ambos hacen referencia a una entidad concreta creada a partir de una clase.
>
> En el siguiente ejemplo, decimos que `l` es una instancia de la clase `list`:
>
> ```
> l = list("abcde")
> ```

4 Creando nuestras propias clases

La definición de una clase, que crea un nuevo tipo de dato, define los atributos de un objeto (datos que representan estado) y las cosas que este objeto puede hacer (funciones que representan comportamiento).

Para definir una nueva clase en Python se usa la sentencia `class` seguida del nombre de la clase. Veamos el siguiente ejemplo comentado línea a línea.

```
class Budin:
    def __init__(self):
        print("Creando nuevo budin")
```

Línea 1

La sentencia `class` indica el inicio de la definición de una clase. A continuación se escribe el nombre de la clase y, al final, los dos puntos (`:`), que señalan el comienzo del bloque donde se definen sus atributos y funciones (llamados métodos).

Línea 2

Por lo general, lo primero que se define dentro de una clase es su método de inicialización, siempre llamado `__init__`.

Línea 3

Esta línea se ejecuta cada vez que se crea un nuevo objeto de la clase. En este caso, imprime un mensaje en pantalla como parte del proceso de inicialización.

La definición de una clase no crea ningún objeto por sí misma; simplemente construye el molde o plantilla a partir del cual se podrán crear objetos más adelante.

Para crear un objeto a partir de una clase, es necesario "llamar" a la clase, lo que genera una nueva instancia de esa clase.

```
budin1 = Budin()
```

```
Creando nuevo budín
```

La variable `budin1` representa un objeto de la clase `Budin`. En otras palabras, hemos creado, al menos en código, un budín.

Como las clases son reutilizables, podemos crear tantos budines como queramos, simplemente creando nuevas instancias de la clase.

```
budin2 = Budin()
```

```
Creando nuevo budín
```

Al imprimir estos objetos, obtenemos una representación generada automáticamente por Python. En ella podemos ver que ambos son instancias de la clase `Budin`, aunque también queda claro que ocupan ubicaciones distintas en memoria. Esto confirma que se trata de objetos distintos, aunque provengan de la misma clase.

```
print(budin1)
print(budin2)
```

```
<__main__.Budin object at 0x7f8854339220>
<__main__.Budin object at 0x7f8854338530>
```

4.1 Parámetros de inicialización

También es posible pasar argumentos al momento de inicializar un objeto. Previamente, tenemos que agregar los parámetros necesarios en el método de inicialización.

```
class Budin:
    def __init__(self, base):
        print(f"Creando nuevo budín de {base}")
```

Línea 2

El método `__init__` ahora recibe dos parámetros:

Línea 3

El valor de `base` se utiliza para mostrar un mensaje de inicialización personalizado, indicando la base del budín.

Gracias a esto, ahora podemos crear budines con distintas bases, como vainilla o chocolate, entre otras.

```
budin1 = Budin("vainilla")
budin2 = Budin("chocolate")
```

```
Creando nuevo budín de vainilla
Creando nuevo budín de chocolate
```

Sin embargo, podemos observar que ninguna de estas dos instancias recuerda algo relacionado con la base con la que fue inicializado.

```
budin1.base
```

```
AttributeError: 'Budin' object has no attribute 'base'
```

```
budin2.base
```

```
AttributeError: 'Budin' object has no attribute 'base'
```

Si queremos que nuestros objetos puedan recordar datos, necesitamos comprender cómo se utilizan los **atributos**.

> **i** Instanciación de un nuevo objeto
>
> La instanciación es el proceso mediante el cual se crea un objeto a partir de una clase. La sintaxis general es la siguiente:
>
> ```
> <objeto> = <NombreClase>(<argumentos opcionales>)
> ```
>
> Aunque al principio pueda parecer poco familiar, en realidad hemos estado instanciando objetos desde los primeros pasos que dimos en Python. Por ejemplo, al escribir `list("hola")`, estamos creando un objeto de la clase `list` a partir de la cadena `"hola"`. Del mismo modo, con `range(10)` craemos un objeto de la clase `range`.

5 Atributos

Los atributos son variables asociadas a un objeto que representan su estado.

En Python, los atributos no requieren ninguna sintaxis especial para ser declarados. Simplemente se crean dentro de un método (generalmente en `__init__`) usando la notación `self.<atributo>`.

En nuestro ejemplo de la clase `Budin`, podemos hacer:

```
class Budin:
    def __init__(self, base):
        self.base = base
```

Línea 3

El valor de `base` que se pasa al momento de crear la instancia se asigna al atributo `base` del objeto.

Aunque la representación automática del objeto no cambia, ahora el objeto tiene **estado**: puede "recordar" la base con la que fue inicializado.

```
budin1 = Budin("vainilla")
budin1
```

```
<__main__.Budin at 0x7f8854339c70>
```

```
budin1.base
```

```
'vainilla'
```

En este caso, la variable `base` es una **variable de instancia** o **atributo de instancia**. Estas variables existen solo dentro del objeto que las contiene, y no afectan a otras instancias de la misma clase. Así, distintos objetos de una misma clase pueden tener valores diferentes en sus atributos.

```
budin2 = Budin("chocolate")
print(budin2.base)
print(budin1.base)
```

```
chocolate
vainilla
```

Los atributos también pueden definirse o modificarse fuera de los métodos de la clase.

Por ejemplo, es posible cambiar el valor de un atributo simplemente asignándole un nuevo valor utilizando la instancia:

```
budin1.base = "marmolado"
```

```
budin1.base
```

```
'marmolado'
```

Y también se puede asignarle un valor a un nuevo atributo:

```
budin1.cobertura = "chocolate"
print(budin1.base, budin1.cobertura, sep=", ")
```

```
marmolado, chocolate
```

Este nuevo atributo existe únicamente en la instancia a la que fue asignado (`budin1`). Es decir, `budin2` no tiene un atributo llamado `cobertura`, ya que no fue definido durante su inicialización ni se le asignó más adelante.

```
budin2.cobertura
```

```
AttributeError: 'Budin' object has no attribute 'cobertura'
```

En Python, los atributos de instancia son independientes entre objetos: si un atributo no se define explícitamente en una instancia, simplemente no existe en ella.

5.1 Atributos opcionales

Aunque Python permite crear nuevos atributos fuera del proceso de inicialización de un objeto, esto no significa que sea una práctica recomendable en todos los casos.

Asignar un atributo directamente a una instancia específica puede generar inconsistencias: terminamos con objetos de la misma clase que no comparten la misma estructura de atributos. Esto puede dificultar la lectura del código y producir errores si intentamos acceder a un atributo que no existe en todas las instancias.

Una alternativa más clara y segura es definir atributos opcionales dentro del método `__init__`, asignándoles un valor por defecto como `None`. De esta forma, todas las instancias tendrán los mismos atributos, aunque algunos puedan no tener un valor definido inicialmente.

```
class Budin:
    def __init__(self, base, cobertura=None):
        self.base = base
        self.cobertura = cobertura

budin1 = Budin("vainilla", "chocolate")
budin2 = Budin("chocolate")
```

```
print(budin1.base, budin1.cobertura, sep=", ")
print(budin2.base, budin2.cobertura, sep=", ")
```

```
vainilla, chocolate
chocolate, None
```

Con este enfoque, ambos objetos tienen los mismos atributos (`base` y `cobertura`), lo que mantiene la consistencia entre instancias de la clase. En el caso de `budin2`, como no se especificó ninguna cobertura al crear el objeto, el valor de `cobertura` es `None`, lo cual indica que ese budín no tiene ninguna cobertura.

6 Métodos

Ahora que comprendemos cómo los datos definen el **estado** de un objeto, el último concepto que nos falta abordar son las **acciones** que un objeto puede realizar.

Para llevar a cabo acciones, los objetos utilizan **métodos**.

Los métodos son **funciones asociadas a una clase determinada**, y permiten que los objetos de la clase realicen operaciones o modifiquen su propio estado.

Por ejemplo, el método `upper` es un método propio de los objetos de tipo `str` (cadenas de caracteres) y podemos invocarlo de la siguiente manera:

```
"Rosario".upper()
```

```
'ROSARIO'
```

Pero no podemos llamarlo sobre una lista:

```
["Rosario", "Santa Fe"].upper()
```

```
AttributeError: 'list' object has no attribute 'upper'
```

En el caso de las clases creadas por nosotros, los métodos no son más que funciones definidas dentro de la clase.

A diferencia de las funciones normales, todos los métodos deben tener al menos un parámetro especial, llamado `self` por convención, que representa a la instancia sobre la que se está llamando el método.

Gracias a `self`, los métodos pueden acceder y modificar los atributos del objeto:

```
class Budin:
    def __init__(self, base, cobertura=None):
```

```
        self.base = base
        self.cobertura = cobertura

    def mostrar_info(self):
        print("Budin")
        print(f" - Base: {self.base}")
        print(f" - Cobertura: {self.cobertura}")
```

```
budin = Budin(base="Vainilla", cobertura="Chocolate")
budin
```

```
<__main__.Budin at 0x7f885433b440>
```

Para ejecutar el método, se lo llama de la misma forma que a cualquier otro método.

```
budín.mostrar_info()
```

```
Budin
 - Base: Vainilla
 - Cobertura: Chocolate
```

```
budin2 = Budin(base="Limon", cobertura="Chocolate blanco")
budin2.mostrar_info()
```

```
Budin
 - Base: Limon
 - Cobertura: Chocolate blanco
```

6.1 Métodos que modifican estado

El método `mostrar_info` no altera el estado del objeto: simplemente muestra un resumen de su estado actual.

Sin embargo, como los métodos acceden al objeto mediante `self`, también pueden **modificar su estado**.

En el siguiente ejemplo, el método `quitar_cobertura` elimina cualquier cobertura que pueda tener nuestro budín.

```
class Budin:
    def __init__(self, base, cobertura=None):
        self.base = base
        self.cobertura = cobertura
```

```
    def mostrar_info(self):
        print("Budin")
        print(f" - Base: {self.base}")
        print(f" - Cobertura: {self.cobertura}")

    def quitar_cobertura(self):
        self.cobertura = None

budin = Budin("Chocolate", "Chocolate")
budin.mostrar_info()
print("")
budin.quitar_cobertura()
budin.mostrar_info()
```

```
Budin
 - Base: Chocolate
 - Cobertura: Chocolate

Budin
 - Base: Chocolate
 - Cobertura: None
```

6.2 Métodos que reciben argumentos

Los métodos `mostrar_info` y `quitar_cobertura` no reciben argumentos, a lo sumo usan los datos ya almacenados en el objeto.

También es posible definir métodos que acepten argumentos, lo que permite realizar distintas operaciones y, por ejemplo, modificar el estado interno del objeto.

A continuación, incorporamos a la clase un nuevo método que permite actualizar la cobertura del budin.

```
class Budin:
    def __init__(self, base, cobertura=None):
        self.base = base
        self.cobertura = cobertura

    def mostrar_info(self):
        print("Budin")
        print(f" - Base: {self.base}")
        print(f" - Cobertura: {self.cobertura}")

    def cambiar_cobertura(self, cobertura):
        self.cobertura = cobertura

budin = Budin("Chocolate", "Chocolate")
```

```
budin.mostrar_info()
print("")
budin.cambiar_cobertura("Chocolate blanco")
budin.mostrar_info()
```

```
Budin
 - Base: Chocolate
 - Cobertura: Chocolate

Budin
 - Base: Chocolate
 - Cobertura: Chocolate blanco
```

6.3 Métodos que devuelven resultados

Como los métodos son funciones de Python, también pueden devolver un resultado.

Por ejemplo, el método `tiene_cobertura` retorna `True` cuando el budín tiene asignada alguna cobertura, sin importar cuál sea.

```
class Budin:
    def __init__(self, base, cobertura=None):
        self.base = base
        self.cobertura = cobertura

    def mostrar_info(self):
        print("Budin")
        print(f" - Base: {self.base}")
        print(f" - Cobertura: {self.cobertura}")

    def cambiar_cobertura(self, cobertura):
        self.cobertura = cobertura

    def tiene_cobertura(self):
        return self.cobertura is not None

budin = Budin("Vainilla", "Chocolate blanco")
budin2 = Budin("Vainilla")
print(budin.tiene_cobertura())
print(budin2.tiene_cobertura())
```

```
True
False
```

6.4 Métodos que devuelven al objeto

Finalmente, como un método puede devolver cualquier tipo de objeto, también puede retornar la propia instancia sobre la que fue llamado (es decir, la que recibe en `self`).

Este patrón se utiliza con frecuencia cuando se desea permitir el **encadenamiento de métodos**, ya que cada método devuelve el mismo objeto y permite seguir llamando otros métodos sobre él en una sola línea.

```
class Budin:
    def __init__(self, base, cobertura=None):
        self.base = base
        self.cobertura = cobertura

    def mostrar_info(self):
        print("Budin")
        print(f" - Base: {self.base}")
        print(f" - Cobertura: {self.cobertura}")

    def cambiar_cobertura(self, cobertura):
        self.cobertura = cobertura
        return self
```

Línea 13

Luego de actualizar la cobertura, se devuelve a la instancia con la que se llamó al método.

```
Budin("Marmolado").cambiar_cobertura("Chocolate").mostrar_info()
```

```
Budin
 - Base: Marmolado
 - Cobertura: Chocolate
```

Como el método `cambiar_cobertura` devuelve la propia instancia, es posible encadenar su llamada con otros métodos de la clase, como por ejemplo `.mostrar_info`.

7 Ejemplo final: Cuenta bancaria

Para concluir este apunte, vamos a implementar una clase que represente de forma sencilla una cuenta bancaria.

En este modelo simplificado, cada cuenta tendrá tres atributos:

- `titular`: el nombre de la persona dueña de la cuenta
- `saldo`: el dinero disponible en la cuenta
- `moneda`: el tipo de moneda en el que está expresado el saldo (por ejemplo, `"ARS"` o `"USD"`)

Además, la clase contará con métodos que permitan realizar operaciones básicas:

- `depositar`: suma dinero al saldo actual

- `retirar`: descuenta dinero del saldo, si hay fondos suficientes
- `transferir`: mueve dinero de una cuenta a otra
- `resumen`: muestra el estado actual de la cuenta de forma clara y legible

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0, moneda="ARS"):
        self.titular = titular
        self.saldo = saldo
        self.moneda = moneda

    def depositar(self, monto):
        self.saldo = self.saldo + monto

    def retirar(self, monto):
        if monto <= self.saldo:
            self.saldo = self.saldo - monto
            return True

        print(f"El saldo es insuficiente ({self.saldo} {self.moneda})")
        return False

    def transferir(self, other, monto): # 'other' es tambien una
'CuentaBancaria'
        if self.moneda != other.moneda:
            print(f"Las cuentas usan monedas distintas ({self.moneda} vs
{other.moneda})")
            return False

        if self.retirar(monto):
            other.depositar(monto)

        return True

    def resumen(self):
        print("Cuenta bancaria")
        print(f" - Titular: {self.titular}")
        print(f" - Saldo: {self.saldo}")
        print(f" - Moneda: {self.moneda}")
```

```
cuenta_A = CuentaBancaria("Guido Van Rossum", saldo=20000, moneda="ARS")
cuenta_B = CuentaBancaria("Ross Ihaka", saldo=8000, moneda="ARS")
cuenta_A.resumen()
print("")
cuenta_B.resumen()
```

```
Cuenta bancaria
 - Titular: Guido Van Rossum
```

```
 - Saldo: 20000
 - Moneda: ARS

Cuenta bancaria
 - Titular: Ross Ihaka
 - Saldo: 8000
 - Moneda: ARS
```

```
cuenta_A.retirar(2500)
```

```
True
```

```
cuenta_A.resumen()
```

```
Cuenta bancaria
 - Titular: Guido Van Rossum
 - Saldo: 17500
 - Moneda: ARS
```

En el caso de las transferencias, no solo se modifica el estado de la cuenta desde la que se realiza la operación (`cuenta_A`), sino también el estado de la cuenta que recibe el dinero (`cuenta_B`).

```
cuenta_A.transferir(cuenta_B, 10000)
```

```
True
```

```
cuenta_A.resumen()
```

```
Cuenta bancaria
 - Titular: Guido Van Rossum
 - Saldo: 7500
 - Moneda: ARS
```

```
cuenta_B.resumen()
```

```
Cuenta bancaria
 - Titular: Ross Ihaka
 - Saldo: 18000
 - Moneda: ARS
```

```
cuenta_B.retirar(20000)
```

```
El saldo es insuficiente (18000 ARS)

False
```

> **i** ¿Sabías que ...? Sobre `__init__`
>
> - `__init__` no lleva una sentencia `return`. Su propósito no es devolver datos, sino inicializar el objeto.
> - `__init__` no crea la instancia (eso lo hace Python antes); su función es inicializar el estado del objeto recién creado.
> - `__init__` no es obligatorio: una clase puede funcionar perfectamente sin definirlo, aunque en ese caso no se podrán establecer valores iniciales personalizados al instanciar.

> **⚠** Sobre `self`
>
> Nunca se debe pasar explícitamente un valor para `self`, Python lo hace automáticamente.